Week 5: Optimization

What optimizer.step() Actually Does

Live Coding Session — Week 5

Gradient Descent: Three Flavors

How many samples do you use to estimate the gradient?

Batch GD

Uses	**ALL data per step**
Gradient	Smooth, accurate
Speed	Slow per step
Memory	Entire dataset in RAM

Smooth convergence

Stochastic GD

Uses	**1 sample per step**
Gradient	Very noisy
Speed	Fast per step
Memory	1 sample

Noisy but explores well

Mini-Batch GD

Uses	**32–256 samples per step**
Gradient	Best of both worlds
Speed	GPU-parallelizable
Memory	Manageable

Smooth-ish + fast = default

In practice: Mini-batch GD is the default. PyTorch's DataLoader + optimizer.step() = mini-batch GD. That's what you've been doing in weeks 1–4.

Learning Rate: The #1 Hyperparameter

Too low = crawls. Too high = diverges. Just right = converges.

The update rule: $\theta = \theta - lr \times \nabla L(\theta)$

θ = model parameters lr = step size ∇L = gradient (which direction to go)

Math	PyTorch	What it does
$\nabla L(\theta)$	`loss.backward()`	Compute gradients
$\theta = \theta - lr \cdot \nabla L$	`optimizer.step()`	Apply the update
(reset)	`optimizer.zero_grad()`	Clear old gradients

SGD → Momentum → Adam: The Evolution

Each optimizer fixes a problem with the previous one

SGD
- $\theta = \theta - lr \cdot \nabla L$
- Simple but slow
- Gets stuck in ravines

Momentum
- $v = \beta v + \nabla L$
- + "velocity" accumulates
- Rolls through ravines

Adagrad
- $lr / \sqrt{(\Sigma g^2)}$
- + per-param learning rate
- LR decays too fast

RMSProp
- + exponential decay
- Fixes Adagrad decay
- Adaptive but no momentum

Adam ★
- Momentum + RMSProp
- + bias correction
- Default for most tasks

AdamW
- + proper decay
- State of the art

Two independent ideas combined:

Momentum (velocity)
Accumulate past gradients for direction
Like a ball rolling downhill — builds speed

+

Adaptive LR (per-parameter)
Scale step size by gradient history
Frequent features → smaller steps

Your default: `optim.Adam(model.parameters(), lr=0.001)` — this is what you've been using all along. Now you know what's inside.

Today's Coding Session

~12 min

GD Variants

BGD vs SGD vs Mini-batch on linear regression. Learning rate sensitivity — watch it diverge.

~15 min

Optimizer Showdown

SGD, Momentum, Adam on the Rosenbrock surface. Then train a neural net with each.

~8 min

PyTorch Mapping

Our from-scratch optimizers vs torch.optim. Same results, less code.

+ 10 min Q&A at the end